Solution: Moving Average from Data Stream

Problem Statement

Given a stream of integers and window size `n`, calculate the moving average of all the integers of the sliding window.

Implement the `Solution` class:

- `MovingAverage(int size)` Initializes the object with the size of the window size, which is `n`.
- `double next(int x)` Calculates and returns the moving average of the last `n` values of the integer stream.

Examples

- **Example 1:**
  - **Input**: `["MovingAverage", "next", "next", "next"]`, `[[2], [1], [10], [3]]`
  - **Expected Output**: `[null, 1.0, 5.5, 6.5]`
  - **Justification**: The moving average of the last 2 numbers. Initially, it's 1. Then (1+10)/2 = 5.5, and (10+3)/2 = 6.5.
- **Example 2:**
  - **Input**: `["MovingAverage", "next", "next"]`, `[[4], [5], [15]]`
  - **Expected Output**: `[null, 5.0, 10.0]`
  - **Justification:** First, the average is 5. Then, (5+15)/2 = 10 as we consider the last 4 numbers, but only two numbers are in the stream.
- **Example 3:**
  - **Input**: `["MovingAverage", "next", "next", "next", "next"]`, `[[3], [7], [4], [8], [5]]`
  - **Expected Output**: `[null, 7.0, 5.5, 6.33, 5.67]`
  - **Justification**: Moving average calculations are (7), (7+4)/2, (7+4+8)/3, and (4+8+5)/3.

Solution

To solve this problem, we can use a queue to store the most recent 'n' numbers. A queue is perfect for this task as it allows easy addition of new elements and removal of old elements, maintaining the 'window' of the last 'n' elements.

The approach is efficient as it avoids recalculating the sum from scratch for each new element. When a new number is added, we add it to the sum and enqueue it. If the queue exceeds the size 'n', we dequeue an element and subtract it from the sum. This way, we always have the sum of the last 'n' elements, allowing us to quickly calculate the new average.

Step-by-Step Algorithm

- Initialize a `queue` and a variable to store the `sum` of the last 'n' elements.
- When a new number arrives:
  - Add the number to the `queue`.
  - Add this number to the `sum`.
  - If the queue's size exceeds `n`:
    - `Dequeue` the oldest number from the queue.
    - `Subtract` this number from the sum.
  - Calculate the average by dividing the `sum` by the minimum of `n` or the `queue's current size`.

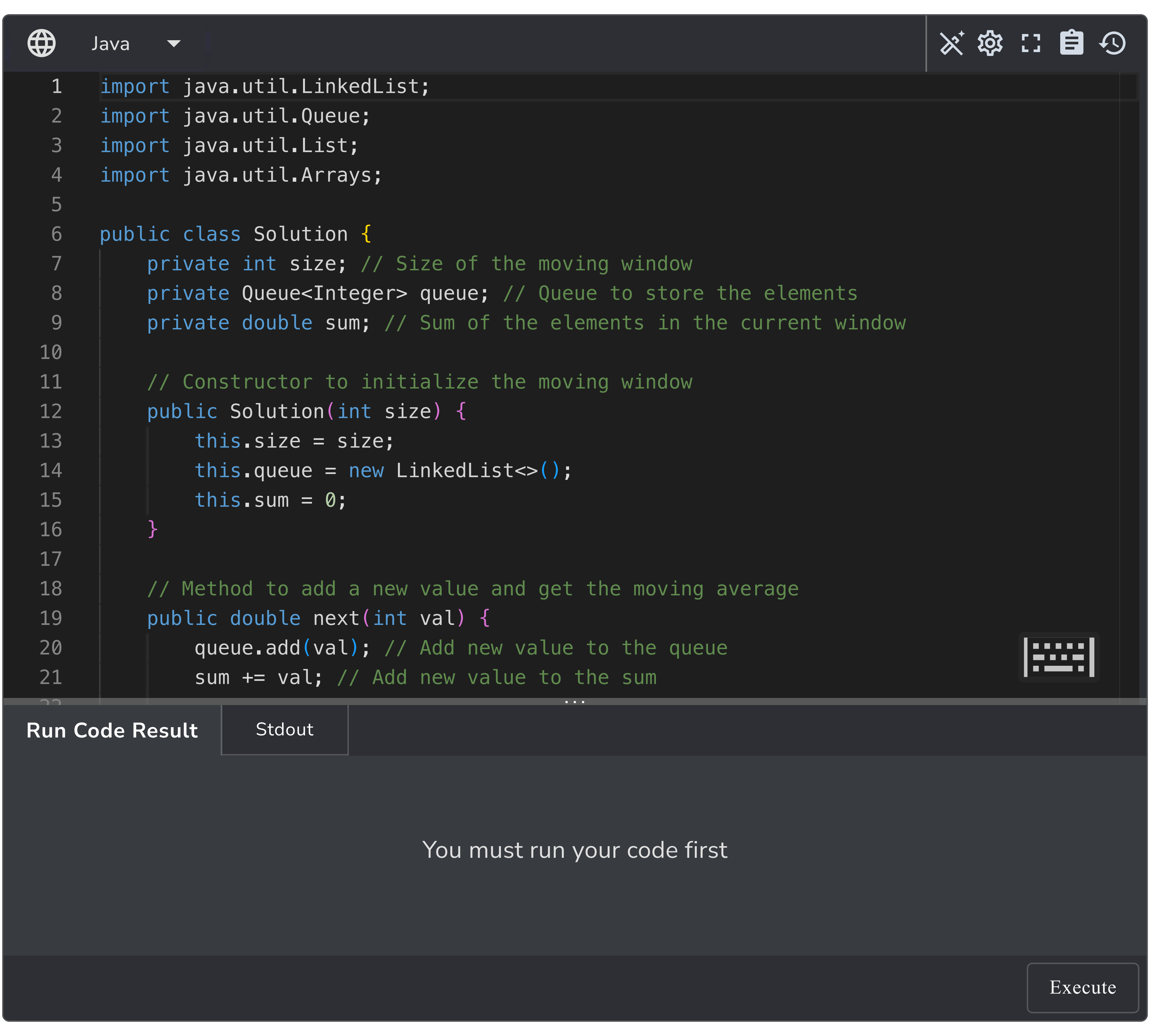
Algorithm Walkthrough

Let's use **Example 3**:
`["MovingAverage", "next", "next", "next", "next"], [[3], [7], [4], [8], [5]]`

1. **Initialization:**
   - Create a queue to represent the moving window of size 3.
   - Initialize the sum to 0.
2. **First Value -** `next(7)`:
   - **Enqueue '7'**: Add '7' to the queue. Queue now contains [7].
   - **Update Sum:** Sum = 0 + 7 = 7.
   - **Calculate Average:** Average = Sum / Queue Size = 7 / 1 = 7.0.
   - **Queue State:** [7], **Sum**: 7.
3. **Second Value -** `next(4)`:
   - **Enqueue '4'**: Add '4' to the queue. Queue now contains [7, 4].
   - **Update Sum:** Sum = 7 + 4 = 11.
   - **Calculate Average:** Average = Sum / Queue Size = 11 / 2 = 5.5.
   - **Queue State:** [7, 4], **Sum:** 11.
4. **Third Value -** `next(8)`:
   - **Enqueue '8'**: Add '8' to the queue. Queue now contains [7, 4, 8].
   - **Update Sum:** Sum = 11 + 8 = 19.
   - **Calculate Average:** Average = Sum / Queue Size = 19 / 3 ≈ 6.33.
   - **Queue State:** [7, 4, 8], **Sum:** 19.
5. **Fourth Value -** `next(5)`:
   - **Queue Exceeds Size:** Before adding '5', the queue size equals the window size (3).
   - **Enqueue '5' and Dequeue '7'**:
     - Add '5' to the queue. Queue temporarily becomes [7, 4, 8, 5].
     - Remove '7' from the queue as it exceeds the window size. Queue now contains [4, 8, 5].
   - **Update Sum:** New Sum = Previous Sum - Dequeued Value + New Value = 19 - 7 + 5 = 17.
   - **Calculate Average:** Average = Sum / Queue Size = 17 / 3 ≈ 5.67.
   - **Queue State:** [4, 8, 5], **Sum**: 17.

Through each `next` operation, the queue dynamically updates to hold only the last 3 values, and the sum is adjusted to reflect the current window's total value, allowing the calculation of the moving average at each step.

Code



```java
import java.util.LinkedList;
import java.util.Queue;
import java.util.List;
import java.util.Arrays;

public class Solution {
    private int size; // Size of the moving window
    private Queue<Integer> queue; // Queue to store the elements
    private double sum; // Sum of the elements in the current window

    // Constructor to initialize the moving window
    public Solution(int size) {
        this.size = size;
        this.queue = new LinkedList<>();
        this.sum = 0;
    }

    // Method to add a new value and get the moving average
    public double next(int val) {
        queue.add(val); // Add new value to the queue
        sum += val; // Add new value to the sum
```

Complexity Analysis

- **Time Complexity**:
  - The time complexity is O(1) for each next operation, as adding, removing, and calculating the average are constant time operations.
- **Space Complexity**:
  - The space complexity is O(n), where n is the size of the moving window, as we need to store up to n elements.

← Previous
Moving Average From Data Stream (Easy)
Next →
Move Zeroes (Easy)
☐ Mark as Completed